Mechanising Count-One Ownership for Array Updates in Lean 4

Gavin Mendel-Gleason
Scidonia Limited
gavin@scidonia.ai

2 November 2026

Lean statements are quoted from the development; hypotheses are abbreviated where the text says so.

Abstract

Reference-counted array updates copy when an extra reference survives. Lean 4 decides the copy at run time, from the heap's count and persistence, so the in-place branch is not visible in the source type. We mechanise a count-one ownership discipline for a recognised fragment of Lean's compiler IR and prove, in Lean 4, (i) that a supported flat array-update fragment, from an accounted state (`Accounted`) with the store-slot fact supplied (`StoreSlots`), preserves a complete accounting of owning roots and incoming heap edges at every state it reaches, (ii) that the emitted extraction pair's count traffic is neutral at its boundary and that restoration takes the exclusive branch, and (iii) that the recognised finite path corresponds to the model. The principal nested theorem composes the finite-path correspondence, the counted extraction/call/restoration protocol at its accounted boundaries, and the counted interpreter's recognised-prefix boundary for one arbitrary level of a finite supported path, from an accounted caller state (`hacc`) and conditional on a child-call contract (`hcall`), and a closed kernel-checked instance inhabits it at the recogniser's own layout; the flat recognised write is the companion result and takes no accounting premise. The claim stops at that boundary: the emitted recursive callee, trace-level agreement beyond the counted prefix, extraction/restoration event coverage, C semantics, replay and address identity, and the source correspondence remain open, and neither model's count is a physical count.

1 Introduction

Reference-counted array updates copy when the count at the write exceeds one. Lean 4 decides the branch at run time, from the heap's count and persistence [3, 9, 10], so the same source program can copy or mutate depending on sharing that its type does not mention. The question is whether a checked discipline can make the in-place branch a property of a recognised program fragment rather than of the current heap.

The obvious argument—"the variable is owned, so the count is one"—fails. `Inv` equates the stored count with the context's multiplicity, but does not count owning roots: `bindOwned` preserves `Inv` while leaving the multiplicity unchanged, and `alias_not_accounted` exhibits the two-owner state. The first repair is a representation: `Accounted support H env c` requires a duplicate-free, sound, complete and bound support, the live-cell domain, and the equation `rc = IncomingCount + EnvOwnerCount`. At count one it yields `count_one_dichotomy` and `at_most_one_owner`; at the initializer it is established by `Accounted.initCtx` (Section 3).

The second obstacle is the emitted code. The take-out builds a pair whose fields are the parent and the extracted child; the projections increment both; freeing the unique pair releases both edges.

The counts return to their entry values only because the pair is freed: `pair_accounting` proves the boundary, and the no-free control leaves a count of two. Restoration has two dispositions; `putBack_exclusive_of_wellFormed` derives the mutate branch from the protocol boundary and the child-call boundary, not from an assumed final state (Section 5).

The third obstacle is the compiler connection. The checker is about a model; the paper relates the model to the operation trace the recogniser emits (`project_sound`, `recognized_write_in_place`) and to the finite nested path (`index_path_simulates_model`), with the child contract derived at every level from one supplied leaf. The composed statements are three results with different premises. `principal_write` composes the emitted, model and runtime-model claims under the recogniser's verdict, the layout, `EntryInv`, `Agrees`, the cell and bounds facts, and `Corr`; it takes *no accounting premise*. Adjacent to it, `Prefix.toModel_supported` places the projected flat model in `Supported`, so `supported_run` preserves `Accounted` from an accounted state given `StoreSlots` for that model—a fact supplied at `execSet`, not generally for `p.toModel`. `principal_supported_path` composes the finite-path correspondence, the counted chain and the counted-prefix boundary from an accounted caller state (`hacc`) and conditional on `CountedChildCall` (`hcall`); the emitted recursive callee is not captured, and the paper says so wherever the claim appears (Section 5.5).

The take-away is a chain of three proved relations—complete accounting, a counted protocol at its accounted boundaries, and a fragment correspondence—not a type system for Lean. The boundary is part of the result: C semantics, replay and address identity, the emitted callee, trace-level agreement beyond the counted prefix, extraction/restoration event coverage, whole-path non-growth, and the source correspondence remain open (Section 8).

Three contributions.

- **Complete accounting and preservation for the supported flat fragment.** `Accounted` with its support, domain and count fields, `Supported`, `supported_run` (from `Accounted` plus the supplied `StoreSlots`), `supported_store_exclusive`, `storeSlots_init`, and the independence and negative results `alias_not_accounted`, `aliasEntryInv` and `accountedNotEntryInv`.
- **A counted extraction, call and restoration protocol at its accounted boundaries.** `pair_accounting`, `pair_traffic_neutral`, `no_free_components_not_exclusive`, `counted_takeOut`, `counted_restore`, `counted_nested_chain`, `putBack_exclusive_of_wellFormed`, `HeapCompat`, and the pair-live refusal `pairPhase_refused`.
- **Fragment correspondence and a composed guarantee with a closed instance.** `project_sound`, `recognized_write_in_place`, `index_path_simulates_model`, `childContracts_of_path`, `principal_write`, `principal_supported_path` and `principal_instance`, with the `CountedChildCall`/N10 boundary explicit.

The discipline is a deep embedding: `Code` and `NCode` are explicit syntax interpreted by `run`/`runN`, and `Ctx`, `Heap`, `check` and the theorems are ordinary Lean definitions. The source is Lean, lowered to LCNF; a recognised prefix is mapped to a model program. The guarantee is conditional on `EntryInv`, `Agrees`, the contracts A1–A12/N1–N12 and `Corr`; it does not extend to the linked C runtime beyond the measured gate.

2 Setting and the running example

Lean 4 compiles through a sequence of IRs. The last LCNF phase inserts reference-count operations, and the backend emits C. The runtime decides an array update with `lean_is_exclusive`: count

one and non-persistent means in place; every other case copies. In this setting the object is single-threaded; count one and non-persistence characterise this test and are not a claim about every Lean runtime mode. The development pins the toolchain, the sibling revision and the captured LCNF artifacts, so every claim names the code it is about.

The problem is not to implement reference counting, but to know statically that the count at a particular write is one. Lean's source type system has no linearity or uniqueness; adding one would be a language change. We therefore state the property over the compiler's code and prove the connection, rather than extend the type system.

2.1 Model programs

The checker does not run on Lean source or on LCNF directly. Its programs are two small inductive types, interpreted over the model heap and environment; the compiler connection is a mapping from the recognised LCNF to these values.

A `LetF` is one of `lit n`, `size a`, `isShared x` or `checkAlias x`.

```
inductive Code where
  | let (x : Var) (v : LetF) (k : Code)
  | inc (x : Var) (n : Nat) (k : Code)
  | dec (x : Var) (n : Nat) (k : Code)
  | check (x : Var) (k : Code)
  | setArr (x : Var) (i : Var) (v : Var) (k : Code)
  | ret (x : Var)

inductive NCode where
  | base (c : Code)
  | takeOut (a i fst snd : Var) (e : Ref) (k : NCode)
  | putBack (a e i result : Var) (k : NCode)
  | callAt (state path x result : Var) (body : NCode) (k : NCode)
  | ret (x : Var)
```

A `Code` is the straight-line array-write fragment; an `NCode` adds extraction, restoration and the recursive call. Variables are `Nat`, references are `Nat`, `Val` is a reference, a scalar or erased, and a `Heap` maps references to objects with a field array, a count and a persistence flag. `run`/`runN` execute these values and `check`/`checkN` are the checkers; the soundness theorems are about this semantics. They are neither Lean source nor LCNF: `projectPrefix` and `projectNested` walk an LCNF value, `Prefix.toModel` and `indexModelTerm` read a `Code`/`NCode` off the recognised operands, and `execAST`/`execNestedAST` execute the recognised trace. The correspondence theorems relate that execution to the model run.

2.2 The nested write

The source is the array case of `Layout5.setAt` in the sibling. It takes the element out, checks the extracted focus, recurses on it and puts the result back:

```
| _, .index i q, y, s =>
    if h : i < s.size then
      match takeOutAt (assertExclusiveArray openSite s) i h (stateDefault _) with
      | ⟨(sib, focus), hbound⟩ =>
        plugAt sib i hbound (setAt q y (assertExclusive structSite focus))
    else s
```

The emitted C for this arm is the sequence the development names: a `lean_is_exclusive` test for the array, the `takeOut` exchange (`lean_array_fget`, `lean_array_fset`, the pair allocation), the child check on the extracted element, the recursive call and the `lean_array_fset` put-back. The recogniser `projectNested` accepts the `Path.index` arm, `projectPair` accepts the two pair arms, and `execNestedAST` executes the accepted trace. The model term is `Path.build path value body ls`; `checkN` checks it from `initCtx`.

The example separates what the tool computes from what a client supplies. The compiler computes the LCNF and the C; `projectNested` computes the prefix; `Prefix.toModel`/`indexModelTerm` compute the model program; `checkN` computes the checker result. The client supplies the source declaration, the `Path` and the take-out name, and the proofs of `Agrees`, `LevelMatch`, `PathContract` and `EntryInv` (the path/slot and entry facts the checker cannot see). The recursive call is a contract on an admitted model body: the child contract is derived at every level by the finite-path derivation, with one leaf computation supplied. The depth-two witness instance in `OwnershipTest/PathRecursion.lean` takes model index 1 at both levels and leaf `.base (.setArr 9 5 2 (.ret 9))`; its tip slot 1 goes from scalar 6 to scalar 1, the root and child slot 1 are restored to references 1 and 3, and the root's sibling slot 0 keeps scalar 5. A deeper level that takes its index by re-projecting the continuation is outside the model, because the flat environment cannot express the emitted per-invocation index; the emitted call's meaning (N10) is not captured. `OwnershipTest/PathConsolidation.lean` supplies a depth-three leaf in `takeOut`/`putBack` form as a supplied-leaf witness, not an instance of the correspondence.

3 The accounting invariant

The model heap, its counts and exclusivity are the semantic side of the judgement: `Inv` ties the context's multiplicity to the heap count, and `Exclusive` is count one and non-persistence.

Claim 1 (The raw invariant and the exclusivity predicate).

```
structure Inv (H : Heap) (env : Env) (c : Ctx) : Prop where
  rc_eq         : ∀ r, H.rc r = c.mult r
  owned         : ∀ x r, c.us x = .owned → c.refOf x = some r →
                    env x = some (.ref r) ∧ 1 ≤ c.mult r
  borrow        : ∀ x r, c.us x = .borrow → c.refOf x = some r →
                    env x = some (.ref r) ∧ H.cells r ≠ none
  scalar        : ∀ x, c.us x = .scalar → ∃ n, env x = some (.scalar n)
  nonPersistent : ∀ r o, H.cells r = some o → o.persistent = false
```

`Exclusive H r` means that `r` is a live, non-persistent cell whose count is one.

The displayed invariant relates the heap count to the context multiplicity, but it does not count the environment's owning variables: `Inv.owned` only requires `1 ≤ c.mult r`. `bindOwned` preserves `Inv` without changing `mult`, so two owned variables can denote one count-one cell. The raw invariant therefore does not imply physical uniqueness. The kernel countermodel `alias_not_accounted` constructs exactly that state: `initCtx` has one owner, and binding a second owned variable leaves the multiplicity at one. The initializer's context does satisfy `OwnersAccounted` (`initCtx_accounted`). `CountFidelity` is not a consequence of `Inv`, but it is a consequence of the relation below (`countFidelity`); the residual runtime fidelity is not proved.

Claim 2 (Complete accounting).

```
structure Accounted (support : List Var) (H : Heap) (env : Env) (c : Ctx) : Prop where
  inv              : Inv H env c
  support_nodup    : support.Nodup
  support_sound    : ∀ x, x ∈ support → c.us x = .owned
  support_complete : ∀ x, c.us x = .owned → x ∈ support
  support_bound    : ∀ x, x ∈ support → ∃ r, c.refOf x = some r
  domain           : ∀ r o, H.cells r = some o → r < H.next
  count            : ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c support r
```

The relation is deliberately redundant: the support is a duplicate-free enumeration of every owning variable, each entry is bound to a reference, every live cell lies inside the range the edge count enumerates, and the count equation holds over two independently computed quantities.

```
def IncomingCount (H : Heap) (r : Ref) : Nat :=
  ((List.range H.next).map fun q =>
    match H.cells q with
    | some o => (o.fields.toList.filter (fun v => v = .ref r)).length
    | none => 0).sum

def EnvOwnerCount (c : Ctx) (vars : List Var) (r : Ref) : Nat :=
  (vars.filter (fun x => c.us x = .owned ∧ c.refOf x = some r)).length

def CountFidelity (H : Heap) (c : Ctx) (vars : List Var) : Prop :=
  ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c vars r
```

`mult_eq` reads the equation through `Inv.rc_eq`; `count_one_dichotomy` splits count one into one owning root with no incoming edge, or no owner with exactly one incoming edge; `at_most_one_owner` says two owning variables at count one are the same variable; `countFidelity` is the equation in the retained predicate's shape; and `ownersAccounted` recovers the environment-side predicate. The initializer establishes the relation (`initCtx`).

The supported fragment. `Supported c s` is indexed at the pre-state `c`; each constructor states the checker's own rule at `c` and its continuation at the checker's post-state, and carries the certificates the checker cannot see: a fresh alias target and a fresh scalar-`.let` target. The fragment has eight constructors; `.inc` is absent and every `.dec` is the single-count form `.dec x 1`, where `decOne`'s premise is exactly the checker's `n < c.mult r` at `n = 1`. From `Supported c s` the checker accepts the program (`supported_check`), which is what lets a preservation theorem follow a fault-free run. The checker itself is unchanged.

The store's slot fact is deliberately absent from the constructors: no context records which value a cell's selected slot holds, so it is the `StoreSlots` premise of `supported_run` rather than a constructor premise. `StoreSlots H env s` is a recursive predicate over the supported program, supplied from the heap and environment alone (it may not assume the invariant): at the store the run reaches, the array cell's selected slot holds a scalar, so the in-place write replaces a value rather than an edge. `storeSlots_init` discharges it for the initializer's measured write; it is not a general `StoreSlots` for `p.toModel`.

4 The checker and preservation

4.1 The judgement and its rules

A context records, for each variable, whether it is owned, borrowed, scalar or dead, which reference it denotes, and the multiplicity of that reference. The checker is a total function from a context and a program to a successor context or failure; the frame stack records open extractions. The rules in Table 1 are ownership obligations: a write is admitted only for an owned handle of multiplicity one, an extraction only with the frame discipline, and a call only with an isolated body that returns the state it was handed.

The object language is untyped: there is no type former for a program, and the checker does not infer one by unification. The typing judgement is $c \vdash \mathit{code} \Rightarrow c'$, defined to hold exactly when `check c code = some c'`; the context is the type of the program point. A context is a usage, reference and multiplicity map; the checker is syntax-directed.

```
inductive U where
  | owned
  | borrow
  | scalar
  | dead

structure Ctx where
  us    : Var → U
  refOf : Var → Option Ref
  mult  : Ref → Nat
```

A frame records the array, index and element of an open extraction; `check : Ctx → Code → Option Ctx` and `checkNF : Ctx → List Frame → NCode → Option Ctx` are total.

The judgement is syntax-directed: each constructor has one rule, the context is an input, and the successor context is computed by `setUs`, `move`, `bindOwned`, `inc` and `dec`. There are no metavariables, no unification and no implicit type arguments in the object language; the only computation is the context update. This is why the discipline makes the count visible at the type level: at the write, the array's handle has usage `owned` and multiplicity one in the context, and the rule refuses every other usage.

Two consequences are worth stating. `dec` is affine, not the last drop: its premise is $n <$ the object's multiplicity, so it consumes a duplicate handle and leaves the object live. And linearity at the write is the side condition `multOf x = 1`, not the usage tag alone.

A derivation. The first example is the two rules applied in sequence. The premise of the second rule is the context the first produces; no type variable is solved.

$$\frac{\text{us } 0 = \text{owned} \qquad \text{multOf } 0 = 1 \qquad c_1 = c_0.\text{move } 0\ 3}{c_0 \vdash \texttt{checkAlias}\ 0 \Rightarrow c_1} \qquad \frac{\text{us } 3 = \text{owned} \qquad \text{multOf } 3 = 1 \qquad \text{us } 1 = \text{us } 2 = \text{scalar} \qquad c_1 \vdash \texttt{ret}\ 3 \Rightarrow c_1}{c_1 \vdash \texttt{setArr}\ 3\ 1\ 2;\ \texttt{ret}\ 3 \Rightarrow c_1}$$

Composing the two gives $c_0 \vdash \texttt{execSet}\ 0\ 3\ 1\ 2 \Rightarrow c_1$, i.e. `check (initCtx 0) (execSet 0 3 1 2) = some ((initCtx 0).move 0 3)`. The checker computes c_1; the derivation is the rule applications.

The frame stack is the local form of the ownership invariant. An extraction opens a frame naming the array, index and element; a put-back closes the matching frame; a call checks its body under an isolated context and continues with the result.

Rule	Obligation	Effect
`let x (lit n)`	—	`x` scalar
`let x (size a)`, `let x (isShared a)`	`a` owned or borrow	`x` scalar
`let x (checkAlias a)`	`a` owned, multiplicity one	`x` is the checked alias; `a` dead
`check x`	`x` owned, multiplicity one	unchanged
`setArr x i v`	`x` owned, multiplicity one; `i`, `v` scalar	exclusive write through `x`
`inc x n`	`x` owned	multiplicity $+n$
`dec x n`	`x` owned; $n <$ multiplicity	multiplicity $-n$; `x` dead
`takeOut a i fst snd e`	`a` owned, multiplicity one; `i` scalar; `e` multiplicity one; `fst`, `snd` dead and distinct; `e` ≠ `a`; no open frame at `a`	push the frame; bind parent and element
`putBack a e i result`	`a`, `e` owned, multiplicity one; `i` scalar; `e` ≠ `a`; `e` ≠ `result`; the frame is `(a,i,e)`; `result` dead	pop the frame; bind the result
`callAt state path x result body`	`state` owned, multiplicity one; `path`, `x` scalar; `result` dead; `CountFree body`; no open frame at `state`; the body is accepted from the isolated context and returns `state`	call the body; bind the result
`ret x`	no frame: `x` owned or scalar; one frame: `x` is the frame's element; deeper: refused	exit

Table 1: The checker's ownership obligations.

4.2 Two examples

Example 1 (A unique write).

```
def execSet (x y i v : Var) : Code :=
  .let y (.checkAlias x) (.setArr y i v (.ret y))

#guard (check (initCtx 0) (execSet 0 3 1 2)).isSome
```

Example 2 (An aliased write is refused).

```
def aliasSet (x y i v : Var) : Code :=
  .inc x 1 (execSet x y i v)

#guard (check (initCtx 0) (aliasSet 0 3 1 2)).isNone
```

The alias increments the handle, so the write's multiplicity-one obligation fails; the same refusal appears at the context level for a heap whose count is two. This is the static counterpart of the runtime copy: the checker rejects the program before it can copy.

4.3 The invariant and soundness

The entry invariant adds the facts a local checker cannot see: the invariant tree, client separation and root completeness. For fresh arrays `initEntryInv` establishes it, and `checkN_sound_of_entryInv` combines it with the checker.

`check_sound` says that an accepted program run from an invariant state finishes in a state that satisfies the invariant and records only exclusive writes. `EntryInv` extends `Inv` with the invariant tree, client separation and root completeness, and `checkN_sound_of_entryInv` adds those facts and `Agrees` to the checker's acceptance, concluding the same invariant and fault-free run for the nested program.

The checker's local rules do not name the heap's path structure; the path theory supplies it, composing agreement through take-out, call and put-back and lifting the checker to every finite path. The extraction and restoration stores are not trace events; their target-cell dispositions are the model facts `checkN_extraction_exclusive` and `checkN_extraction_exclusive_of_entryInv`.

4.4 Preservation through the supported fragment

`Supported` is the counted part of the checker's accepted set. A supported program from an accounted state, with the store-slot premise, runs without a fault, records only exclusive writes, satisfies the complete relation at every state it reaches, and satisfies it at the final state at the support the step arithmetic determines.

Claim 3 (Supported execution preserves accounting).

```
theorem supported_run {c : Ctx} {s : Code} (hs : Supported c s)
    {support : List Var} {H : Heap} {env : Env}
    (hacc : Accounted support H env c) (hslots : StoreSlots H env s) :
    ∃ c' H' env' v ws,
      check c s = some c' ∧
      run s env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧
      Accounted (resultSupport s support) H' env' c' ∧
      AccountedAlong H env c support s
```

`AccountedAlong` is the conclusion, never a hypothesis: at each instruction the relation holds at the interpreter's own post-state, and at each array store the target is exclusive. `supported_store_exclusive` states the store's own pre- and post-state: it is reached with its target

exclusive and its slot scalar, and the post-state is accounted at the checker's unchanged context. `storeSlots_init` supplies the slot for the initializer's measured write.

Three hard cases. The projections temporarily raise both counts because the pair's edge and the projected handle are both live; the free of the unique pair removes exactly those two edges, which is why `pair_accounting` returns to the entry counts and the no-free control does not. The suspended parent remains exclusive across the child call because the child's contract is confined to the child's mutable footprint, not because the parent's exclusivity is assumed: `callAt_frame_confined` takes `AnnotationsIn`, checker acceptance, the return variable, the isolated `Inv` and `TakeOutsOK`; the parent/child separation is a `ChildCall.separated` fact used in the path derivation. And the put-back's mutate branch is derived from the protocol boundary and the child-call boundary, not from an assumed final state.

5 Counted protocol and fragment correspondence

5.1 Recognition and the flat correspondence

A source-level discipline is only useful if it survives lowering. The compiler rewrites the program, inlines helpers and chooses the emitted shape; the recogniser walks the actual LCNF value, pinning operation order, arities, operand identities and branch constructors. It rejects the token-preserving near misses (eleven at the top level, eleven on the nested path, fourteen per pair arm), so the correspondence is about the code we name. No universal rejection theorem is claimed. The accepted grammar fixes the check's argument, the array-set operands and their order, the pair constructor and projections, the two `Path` branches and their join, and the continuation boundary.

`project_sound` gives executed/model equality for the abstract prefix; `EntryInv` and layout yield `recognized_write_in_place`, concluding both give `inPlaceOutcome root i v o H`.

`PrefixOutcome` records the target, result/`setResult` binders, heap, disposition, fault and write trace, discarding the variable map and suffix; its trace preserves each write's exclusive/copy bit, and the flat prefix has the single `(root, true)` event. `execAST_recognized` is decoding adequacy, not a semantics; A1–A12 are the independent meanings, which is why decoding is load-bearing.

5.2 The finite nested path

The model nests take-out, call and put-back per level; the finite-path induction discharges the callee contract from the tail and leaves only the leaf contract and the emitted call's meaning (N10). The flat environment cannot express the emitted per-invocation `oproj[1]` index; that bound is recorded.

`index_path_simulates_model` concludes that the executed nested prefix's `NestedOutcome` is the model run of `Path.build path value body (l :: rest)`, completing with an all-exclusive trace, under the recogniser's verdict, layout, level match, agreement and path contract.

For a finite level path the contract is derived at every level: `childContracts_of_path` takes the level list and one leaf computation and yields the child contract at each level, whose body is the remaining levels' `build`; `childContract_of_path` is the single-step form. The leaf computation (agreement, checker acceptance, exit variable, count-freedom, annotation) is the one supplied function-level fact. The claim is the finite path's syntax, not data-dependent recursion. The emitted callee is not captured: `execNested`'s call rule runs the admitted model body with only the call's state, path and value (N10); a deeper level that re-indexes by re-projecting the continuation is outside the model.

5.3 Counted extraction, call and restoration

The take-out builds a pair of the parent and extracted child; the constructor moves the handles in, the projections increment both, and freeing the unique pair releases both edges. The counts return to their entry values only because the pair is freed: the negative controls show that omitting the free, or retaining an alias, leaves a count of two.

Claim 4 (Pair accounting).

```
theorem pair_accounting {H : Heap} {p c : Ref} {i : Nat}
    (hw : WellFormedEntry H p c i) :
    let B := protocol H p c i
    B.2.rc p = 1 ∧ B.2.rc c = 1 ∧ B.2.cells p ≠ none ∧
      B.2.cells c ≠ none ∧ B.2.cells B.1 = none ∧
      Heap.Exclusive B.2 p ∧ Heap.Exclusive B.2 c ∧
      B.2.rc p = H.rc p ∧ B.2.rc c = H.rc c
```

The no-free control, `no_free_components_not_exclusive`, leaves both components at count two, neither exclusive, and the pair live; `WellFormedEntry` fixes both at count one.

Stage	`EnvOwnerCount` (roots)	`IncomingCount` (edges)	parent rc	child rc	disposition / why
entry	1 / 0	0 / 1	1	1	`WellFormedEntry`: root handle; child in the parent slot
take-out	0 / 0	1 / 1	1	1	`takeOut_transfers`: handles moved into the pair
constructor projections	1 / 1	1 / 1	2	2	`incPairComponents_eq`: pair edge plus projected handle
free unique pair	1 / 1	0 / 0	1	1	`pair_accounting`, `pair_traffic_neutral`: edges released
child call	1 / 1	0 / 0	1	1	`CountedChildCall`: parent suspended; child isolated
put-back	1 / 0	0 / 1	1	1	`putBack_exclusive_of_wellFormed`: mutate; child restored to the slot

Table 2: The counted protocol at an accounted caller entry. The pair-live state is outside `Accounted` (`pairPhase_refused`); intermediate rows are per-stage count effects from the named lemmas, not boundary facts.

`counted_takeOut` consumes the array's cell and selected slot plus the context facts from `checkNF`; it derives the boundary accounting, `HeapCompat`, the pair's freshness and cursor advance. `CountedChildCall` adds one field to the model child-call contract: the caller's `Accounted` at the child's return. `counted_restore` derives the mutate branch and accounted post-state; `putBack_exclusive_of_wellFormed` concludes that branch and `Heap.Exclusive` for both the parent and restored child; `counted_nested_chain` relates the counted post-state to `runN` by `HeapCompat`. The pair-live state is outside `Accounted`: this is a boundary relation, not an every-instruction invariant.

The counted interpreter executes five heap-touching operations; `countedPrefix` restricts `execCounted_protocol_boundary` to the first nineteen, through the pair's release. Its heap is the protocol boundary, `HeapCompat` to the abstract take-out heap, and its non-heap fields (including the write trace) agree with `execNested`'s. Beyond it no trace-level agreement is claimed: `HeapCompat` is not invariant in the counted window, and the restoration lies outside `countedPrefix`. Extraction and restoration stores are not trace events; their dispositions are the model facts `checkN_extraction_exclusive`.

The take-out shape is `projectNested`'s `Path.index` arm; the left/right arms are `projectPair`'s, which reads the model term off the operands. `Ownership.Pair.runP` correspondence is a named next step; the protocol is the accounting model, not that correspondence.

5.4 The runtime model

`OwnershipLcnf.RuntimeModel` is a second operational model, related to `Ownership.Heap` by `Corr`: the correspondence is between two models and binds no physical C object or address. It preserves refs, cells, counts, persistence, fields and cursor, so model exclusivity implies the check returns and the write mutates.

Three statements carry the reading: `checkExclusive_no_abort` (under `Corr` and model exclusivity the check never returns `abort`), `setAt_inPlace_of_entryInv` (the set returns in place, preserves `Corr`, keeps the target and writes the scalar), and `prefix_abort_impossible` (the same at the emitted-semantics boundary). None binds a physical C object or address.

5.5 The composed guarantee

The model, the recognised LCNF and the runtime model are separate semantics. Their composition at the flat recognised write is `principal_write`:

Claim 5 (Flat principal guarantee).

```
theorem principal_write
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size)
    (hc : Corr H R) (ho' : R.cells root = some o') :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
      PrefixOutcome.ofModel (Ownership.run (Prefix.toModel assign p) env H) =
        inPlaceOutcome root i v o H ∧
      ∃ H', Runtime.setAt R root i (.scalar v) = some (.inPlace H' root) ∧
        Corr (inPlaceOutcome root i v o H).heap H'
        -- and the target and field conclusions
```

The proof is the conjunction of `recognized_write_in_place` and `Runtime.setAt_inPlace_of_entryInv`; it takes no accounting premise. `Accounted` is consumed at the initializer by `principal_accounted`, which yields `OwnersAccounted`, `CountFidelity` and the count-one dichotomy; `alias_not_accounted` shows the environment side cannot be omitted. Both heaps in the conclusion are models, so the model's count one is not a physical count. The nested companion is one level of a finite supported path.

Claim 6 (Nested principal guarantee).

```
theorem principal_supported_path
    (hproj : projectNested …) (hacc : Accounted support H mEnv c)
    (hcall : CountedChildCall …) :
    ∃ Hm envm ws Hf, … -- the four groups below
```

`hacc` and `hcall` are shown; the remaining premises and the conclusion groups are abbreviated. The level list, level, index, call scalars, leaf body and counted parameters are parameters, not a fixture.

Group 1 is the recognition-to-model correspondence with the model's completed, copy-free trace. Group 2 is the counted boundary and final accounting, pair release and cursor, both exclusive

targets, the `mutate` disposition, restoration and slot, and post-call `HeapCompat`. Group 3 is the counted-prefix boundary, including the write trace. Group 4 is the child frame and restoration locality. `PathContract` is supplied; `childContracts_of_path` derives the child contract at every level, so the leaf and N10 remain. The child call is `CountedChildCall`, the only termination premise; `OwnershipTest/Principal.lean` inhabits it with every premise supplied. Whole-path non-growth outside the root's footprint is not concluded: it needs the leaf's write account.

`Prefix.toModel_supported` puts the certified flat model in the counted fragment, so `supported_run` applies once `StoreSlots` is supplied; `storeSlots_init` supplies that fact only at `execSet`, not for general `p.toModel`. The results are adjacent, not one theorem.

Contract	Provider and status
`EntryInv`, `Agrees`; A1–A12 / N1–N12	client/implementation: unique tree, separation, completeness; agreement; primitive/layout/path/call signatures; `initEntryInv` for fresh arrays; N10 and the supplied leaf remain; not proved of the C runtime
`OwnersAccounted`, `CountFidelity`	model-layer: both are implied by `Accounted` (`ownersAccounted`, `countFidelity` at its support); the checker's take-out/`dec` certificates are carried by `Supported`; `StoreSlots` is supplied; residual runtime fidelity is not proved
`PathContract`, `CountedChildCall`	client; contract (N10): the level data supplies the leaf, the child contract at every level is derived, and the child call is the admitted model body, not the emitted callee
`Corr`; linked runtime; source correspondence	proved/empirical/assumption: `Corr` is between two models and binds no physical C object or address; the linked gate is evidence, not proof; source correspondence is not attempted

Table 3: Contracts and model-layer predicates, classified by provider.

6 The closed instance and the evidence

6.1 The instance and its controls

`principal_instance` inhabits `principal_supported_path` at the recogniser's layout on a reified transcription of the captured declaration; every premise is supplied, so the conclusion has no hypotheses. It is a 25-conjunct existential; the first group is below and 22 further conjuncts cover the counted boundary, restoration, counted prefix and locality.

```
theorem principal_instance :
    ∃ Hm envm ws Hf,
      NestedOutcome.ofExec assign capturedPrefix … = some (NestedOutcome.ofModel …) ∧
      Nested.runN (Path.build pathVar valueVar body [l]) entryEnv H =
        .done Hm envm (.ref 0) ws ∧ (∀ w ∈ ws, w.2 = true)
      -- and 22 further conjuncts
```

Abbreviated at the first group; the full existential is in `OwnershipTest/Principal.lean`.

The retained-parent control: `retained_parent_not_mutate` shows a count-two parent does not mutate; `retained_parent_copies` records the copy and untouched sibling. `OwnershipTest/PathRecursion.lean` is the depth-two witness; `OwnershipTest/PathConsolidation.lean` supplies a depth-three leaf, not an instance. Both are kernel-checked.

6.2 Evidence

A compiled probe compares the model's predicates with `lean_is_exclusive` and address identity on a corpus including a unique array, a live alias, the alias dropped back to one, a persistent literal and a closure-held array. The closed-pure-array negative lifts a pure closed array to a `___closed`

constant read through `lean_obj_once`, so it is shared and copies although the source looks unique. The gate binds the pinned artifacts, IR shapes, C and header; this is evidence, not equivalence.

Case	Checker/model	Runtime disposition	Probe mode	Captured status
unique array	accepted	exclusive, in place	`differential`	exit 0
live alias (count two)	rejected	shared, copy	`differential`	exit 0
closed-pure-array	not certified	shared, copy	`inline-exact`, `closed-case`	exit 0 unchecked; exit 1 armed
nested `setAt`	model/LCNF accepted	model in place; pair allocation	no probe mode	C/IR gate result

Table 4: Observed at `d330fa0`: flat-probe alignment and nested LCNF/IR gate; probe rows are observations, the nested row a gate result.

The table records observations at `d330fa0`, not a current re-run.

6.3 Theorem map

The table maps declarations and support lemmas to roles; it is not exhaustive.

Declarations	Role
`check_sound`; `checkN_sound_of_entryInv`; `Accounted` (`mult_eq`, `count_one_dichotomy`, `at_most_one_owner`, `countFidelity`); `initCtx`; `initCtx_accounted`; `OwnersAccounted`; `alias_not_accounted`; `aliasEntryInv`; `accountedNotEntryInv`; `Supported`; `supported_check`; `supported_run`; `supported_store_exclusive`; `storeSlots_init`	checker; accounting and independence; preservation with `Accounted` + `StoreSlots`
`project_sound`; `recognized_write_in_place`; `execAST_recognized`; `index_path_simulates_model`; `childContracts_of_path`; `childContract_of_path`	recognition and correspondence
`pair_accounting`; `pair_traffic_neutral`; `boundary_correspondence`; `no_free_components_not_exclusive`; `pairPhase_refused`; `counted_takeOut`; `CountedChildCall`; `counted_restore`; `counted_nested_chain`; `HeapCompat`	counted protocol; N10
`putBack_exclusive_of_wellFormed`; `execCounted_protocol_boundary`; `checkExclusive_no_abort`; `setAt_inPlace_of_entryInv`; `prefix_abort_impossible`; `checkN_extraction_exclusive`; `checkN_extraction_exclusive_of_entryInv`	restoration; counted prefix; runtime model; store targets
`principal_write`; `Prefix.toModel_supported`; `principal_accounted`; `principal_supported_path`; `principal_instance`	flat and nested guarantees

Table 5: Compact theorem map.

7 Related work

Huisinga's 2023 thesis [4] is the closest predecessor. It analyses a model of Lean's IR rather than Lean source or emitted LCNF, enforces uniqueness through a static analysis, and studies the analysis's checker; uniqueness is a static property of the analysis, not the runtime count; there is no compiler link; and soundness and compiler integration remain future work there. This work differs in what is connected: it proves a count-one checker against a model heap and composes that result with the recognised emitted prefix and a runtime model at the flat write, while the counted protocol is isolated at its accounted boundaries. It is not a claim to the first static uniqueness analysis for Lean.

Perceus [9] and Counting Immutable Beans [10] turn ownership into a compilation discipline: an ownership pass inserts counts and reuse, the runtime maintains them, and the correctness argument is for the transformation. This work shares the count-one target but asks a different question:

a checker over a model, with a proved relation between owning roots and incoming heap edges, boundary accounting for the emitted extraction pair, and an exclusive restoration derived from the protocol boundary. Perceus optimises allocation and reuse; `Accounted` specifies when count one is justified.

Linear Haskell [2] makes linearity a type-system property through linear arrows and multiplicities; Clean's uniqueness typing [1] is uniqueness, not linearity. Neither is a count-one checker over an existing compiler IR. Lean's `Array.markLinear` [6] is dynamic: it can copy an array to establish uniqueness and, with `LEAN_ABORT_ON_NONLINEAR`, abort on later nonlinear use. Translation validation [8] checks a particular compiler run, not a static discipline. Verified compilation [7, 5] proves a compiler correct for its source language; it is a different category, and we make no comparison row for it.

System	Analysed object	Enforcement	Count representation	Compiler connection	Proof status
Huisinga 2023 [4]	Lean IR model	static analysis	static uniqueness	none	soundness future work
Perceus / Counting Beans [9, 10]	compiler and runtime	ownership pass	runtime count, reuse	compiler and runtime	transformation correctness
Linear Haskell [2]	source language	type checker	linearity, multiplicities	language extension	type-system guarantee
Clean uniqueness [1]	source language	type checker	uniqueness types	language	type-system guarantee
`Array.markLinear` [6]	Lean array API	dynamic mark/copy/abort	runtime count, flag	runtime facility	runtime test
This work	LCNF fragment and models	external checker	roots and edges	model and runtime-model link	kernel-checked

Table 6: Related guarantees, analysed at their own boundary.

Table 6 separates what is analysed from what is enforced: type systems and compilers act on their host language, run-time facilities act per run, and this development checks the emitted fragment and proves the connection to a model and a runtime model, with the linked C runtime a measured assumption.

8 Limits and conclusion

Non-claim 1.
The C semantics of the primitives, replay-to-backend identity (A10), address identity, free and cascade, and the source correspondence are not formalised or attempted; the emitted recursive callee and continuation (N10) are not captured; the flat environment cannot express the per-invocation index; the leaf body and several heap-shape facts are supplied; persistence, concurrency and callbacks are outside the fragment; and the nested linked runtime is not measured.

The accounting layer is not a consequence of `Inv`: `OwnersAccounted` has a kernel countermodel and explicit `bindOwned`/`dec` preconditions, and the checker's certificates are carried by `Supported`. `CountFidelity` is the count equation carried by `Accounted`, but the residual runtime fidelity is not proved. `StoreSlots` is supplied, and `storeSlots_init` covers only the initializer's measured write; the fragment excludes `.inc` and admits only `.dec x 1`.

The composed statements do not transport accounting. `principal_write` takes no accounting premise and does not conclude one; `principal_accounted` consumes `Accounted` at the initializer. `principal_supported_path` is conditional on the caller's complete accounting (`hacc`) and the child-call contract (`hcall`); its child call is an admitted model body, not the emitted callee, and its

conclusion is the reached in-place store at one level, not whole-path non-growth outside the root's mutable footprint. Beyond the counted prefix no trace-level agreement is claimed. The nested trace omits extraction and restoration stores, but each target cell is proved exclusive by `checkN_extraction_exclusive_of_entryInv`, a separate fact rather than a silent absence. The pair-arms correspondence with `Ownership.Pair.runP` is a named next step.

The recognition claim is scoped to the accepted grammar and the mutation corpus, not a universal rejection theorem. The differential mode compares recorded expectations rather than executing the `Ownership` model. The probe returns two on an ordinary mismatch, but `sealed` returns one and so cannot be distinguished from the exclusivity abort; that path is not exercised. The table's observations are at `d330fa0`, two commits behind the Lean development's head, not a current re-run.

Conclusion. The count-one discipline is proved at the model, the recognised LCNF and the runtime model for the flat write, composed as `principal_write`, which takes no accounting premise. The nested `principal_supported_path` composes the finite-path correspondence, the counted chain and the counted-prefix boundary at one level, from an accounted caller state (`hacc`) and conditional on the child-call contract (`hcall`); beyond that boundary, and the emitted callee, remain open. The result buys a checked count one for a recognised fragment, not a type system or a physical count. Next: the C semantics, replay/backend and address identity, free and cascade, and the emitted callee; soundness is proved, not completeness.

Acknowledgements

AI assistance. The development and this manuscript were produced with substantial AI assistance. Tools and model versions used across the development were not recorded; the model identifier recorded for this manuscript revision is `deepseek/deepseek-flash`. Categories: prose, Lean proofs, code, analysis, literature search. The division of labour between the author and these tools is not recorded; the tools produced the section drafts and theorem statements here and assisted the Lean modules and scripts.

Outputs were checked by the pinned kernel build and axiom audit; a declaration census checks every displayed name against the development, and the statements and claims are checked against the development's declarations. Kernel checking is not human validation: proofs were machine-generated and spot-checked, not checked line by line. Dagstuhl's statement (checked 2 November 2026) requires disclosure beyond proofreading; the conference and publisher policy must be rechecked at submission. Paraphrasing does not remove substantive AI involvement.

References

[1] Erik Barendsen and Sjaak Smetsers. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science*, 6(6):579–612, 1996.

[2] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. In *Principles of Programming Languages (POPL)*, 2018.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE-28*, volume 12699 of *LNCS*. Springer, 2021.

[4] Marc Huisinga. Static uniqueness analysis for the lean 4 theorem prover. Master's thesis, Karlsruhe Institute of Technology, 2023.

[5] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. Cakeml: A verified implementation of ml. In *Principles of Programming Languages (POPL)*, 2014.

[6] Lean FRO. Array.marklinear, 2025. Lean API documentation.

[7] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 2009.

[8] George C. Necula. Translation validation for an optimizing compiler. In *Programming Language Design and Implementation (PLDI)*, 2000.

[9] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. In *Programming Language Design and Implementation (PLDI)*, 2021.

[10] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL)*, 2019.